

Домашнє завдання 05 — Мережі. Частина 2

Автор: Андрій Пономарьов
Середовище: macOS, Docker Desktop (Linux 6.12)
Інструменти: `ifconfig`, `dig`, `curl`, `conntrack`; Claude Code — редактура

Завдання: порахувати для 172.20.0.0/22 адресу мережі, broadcast, діапазон і кількість хостів; нарізати блок на 4 рівні підмережі /24; знайти свою приватну адресу й маску та визначити свою підмережу; перевірити наявність IPv6 і записати висновок. Опціонально: VLSM-план VPC, `conntrack -L`, `dig AAAA` + `curl -6` проти A-запису.

Примітка: робота виконувалась на macOS, де немає `ip addr` та `conntrack` (обидві — з linux-пакетів `iproute2/netfilter`). Використано нативний еквівалент `ifconfig`, а `conntrack` продемонстровано в Docker Desktop (Linux-ядро 6.12).

Оновлення 27.07.2026: топологія мережі змінилась — ноутбук тепер «схований» за новим роутером (GL.iNet), який сам підключається по Wi-Fi до старого Google-роутера, тобто працює каскад із подвійним NAT. Усі практичні перевірки прогнано повторно: моя підмережа тепер 192.168.8.0/24, провайдер той самий (IPOINT, публічна адреса 91.226.253.120); висновок щодо IPv6 не змінився — ба більше, до ноутбука тепер не долітає навіть ULA старого роутера.

Коротко про результат

Пункт	Результат
Блок /22	172.20.0.0–172.20.3.255: 1024 адреси, 1022 адреси хостів
4 рівні підмережі	172.20.0.0/24, 172.20.1.0/24, 172.20.2.0/24, 172.20.3.0/24
Моя підмережа	192.168.8.0/24; адреса хоста — 192.168.8.230
IPv6	лише link-local; ані глобальної адреси, ані маршруту в інтернет немає
Опціонально	VLSM-план, запис NAT у <code>conntrack</code> , порівняння DNS A та AAAA

1. 172.20.0.0/22 — розрахунок

Маска /22 у двійковому вигляді:

```
11111111.11111111.11111100.00000000 = 255.255.252.0
└─── 22 біти мережі ┘ 10 біт ┘
```

Межа маски проходить усередині третього октета, крок блоку в ньому $256 - 252 = 4$, тому мережа накриває третій октет від 0 до 3:

Параметр	Значення	Як отримано
Адреса мережі	172.20.0.0	усі 10 хостових біт = 0
Broadcast	172.20.3.255	усі 10 хостових біт = 1
Діапазон хостів	172.20.0.1 – 172.20.3.254	між мережею і broadcast
Кількість хостів	1022	$2^{10} - 2 = 1024 - 2$

«-2» — бо адреса мережі та broadcast хостам не роздаються. Викладки у двійковому вигляді: <docs/subnet-calc.txt>.

2. Нарізка на 4 × /24

Щоб з /22 отримати 4 підмережі, позичаємо 2 біти з хостової частини ($2^2 = 4$). Кожна /24 — це 256 адрес, з них 254 хостових:

Підмережа	Мережа	Діапазон хостів	Broadcast	Хостів
1	172.20.0.0/24	172.20.0.1 – 172.20.0.254	172.20.0.255	254
2	172.20.1.0/24	172.20.1.1 – 172.20.1.254	172.20.1.255	254
3	172.20.2.0/24	172.20.2.1 – 172.20.2.254	172.20.2.255	254
4	172.20.3.0/24	172.20.3.1 – 172.20.3.254	172.20.3.255	254

Перевірка: $4 \times 256 = 1024$ адреси — рівно весь блок /22, без дірок і перетинів.

3. Моя приватна адреса і підмережа

```
$ ifconfig en0
  inet 192.168.8.230 netmask 0xffffffff broadcast 192.168.8.255
```

Що це означає: моя адреса — **192.168.8.230**, маска `0xfffff00` — це 255.255.255.0, тобто **/24**. Отже я в підмережі **192.168.8.0/24**: хости 192.168.8.1–254, broadcast 192.168.8.255, я — хост **.230**. Діапазон приватний (192.168.0.0/16, RFC 1918) — в інтернет такі адреси не маршрутизуються, назовні мене випускає NAT на роутері. Шлюз **192.168.8.1** (за `route -n get default`) — у тій самій підмережі, тому доступний напряму по L2, без маршрутизації. Стара підмережа 10.20.50.0/24 при цьому нікуди не ділась — вона живе за новим роутером (старий Google-роутер досі відповідає на 10.20.50.1), просто мій ноутбук тепер у власному сегменті нового роутера, за **подвійним NAT**: 192.168.8.0/24 → мережа старого роутера → провайдер. Повний вивід: [docs/ifconfig-output.txt](#).

4. Чи є в мене IPv6

macOS-еквівалент `ip -6 addr:`

```
$ ifconfig | grep inet6
    inet6 ::1 prefixlen 128
    inet6 fe80::eb:9209:def9:7d3c%en0 prefixlen 64 secured
    ... (fe80:: на кожному інтерфейсі)
```

Видно лише два типи адрес — і жодної глобальної:

- `::1` — loopback;
- `fe80::/10` — link-local, автоматичні, працюють лише в межах сегмента.

Глобальної адреси з 2000::/3 немає. Цікаво, що старий роутер роздавав ще й ULA (fd5e:... з fd00::/8 — приватний аналог RFC 1918), але тепер ноутбук сидить у сегменті нового роутера, а той ULA не роздає і чужу не трансліює: IPv6 у моєму сегменті обмежився автоматичним link-local.

Практична перевірка замість test-ipv6.com (сайт джаваскриптовий, тому через його дзеркала):

```
$ curl "https://ipv4.test-ipv6.com/ip/?asn=1"
{"ip":"91.226.253.120","type":"ipv4","asn":"16044","asn_name":"IPORT ... UA"}

$ curl "https://ipv6.test-ipv6.com/ip/"
* Could not resolve host: ipv6.test-ipv6.com

$ curl -g "https://[2a00:1450:4001:c0f::64]/" # IPv6-адреса Google
* Immediate connect fail: No route to host
```

Висновок: у перевірненій конфігурації IPv6-інтернет недоступний: на інтерфейсі немає глобальної адреси з `2000::/3`, а пряме з'єднання повертає `No route to host`. Є лише link-local. Показово, що після перебудови мережі (каскад із двох роутерів, зникнення ULA) публічна адреса і ASN не змінились — `91.226.253.120`, `IPOINT (AS16044)`, — тобто відсутність глобального

IPv6 іде саме від провайдера: скільки NAT-рівнів не додавай, IPv6-префікса згори ніхто так і не видав. Цікава деталь: `curl -6 https://google.com` показує `::ffff:142.250.109.100` — IPv4-mapped адресу, тому цей успішний запит не є доказом IPv6-зв'язності. Повні виводи: [docs/ipv6-check-output.txt](#).

5. Опціонально: VLSM-план VPC (web / app / db)

Вихідний блок — той самий 172.20.0.0/22 (1024 адреси, з них 1022 адреси хостів). Вимоги типового трирівневого застосунку: web-tier наймасштабніший (autoscaling, до ~200 інстансів), app — удвічі менший (~100), db — маленький (~20 реплік). Правило VLSM: нарізаємо від найбільшої підмережі до найменшої, кожна починається на межі, кратній власному розміру, — тоді немає ні дірок, ні перетинів:

Tier	Підмережа	Маска	Діапазон хостів	Broadcast	Хостів	Потреба
web	172.20.0.0/24	255.255.255.0	172.20.0.1 – 0.254	172.20.0.255	254	~200
app	172.20.1.0/25	255.255.255.128	172.20.1.1 – 1.126	172.20.1.127	126	~100
db	172.20.1.128/26	255.255.255.192	172.20.1.129 – 1.190	172.20.1.191	62	~20
резерв	172.20.1.192/26 + 172.20.2.0/23	—	—	—	62 + 510	ріст / друга AZ

Така нарізка спрощує CIDR-правила. В AWS **Network ACL** застосовується до підмережі, а **Security Group** — до ENI/ресурсу; доступ до db-tier на порт 5432 краще дозволити лише з Security Group app-tier. Публічність web-tier і відсутність прямого інтернет-маршруту в db-tier задаються таблицями маршрутизації. У кожній AWS-підмережі резервується 5 адрес, тож у /24 доступна 251 адреса для ресурсів — на вимоги плану це не впливає.

6. Опціонально: conntrack -L під час завантаження сайту

На macOS conntrack немає, тому демонстрація в Docker Desktop: контейнер `dz5-curler` (172.17.0.2) у циклі завантажує `example.com`, а таблицю станів дивимось із `host-netns Linux-VM`:

```
$ conntrack -L | grep "dport=80 " | grep 172.17.0.2
tcp  6 119 TIME_WAIT src=172.17.0.2 dst=172.66.147.243 sport=58756 dport=80
      src=172.66.147.243 dst=192.168.65.3 sport=80 dport=58756 [ASSURED]
```

Що це означає: це і є мій запис трансляції. Перша половина рядка — оригінальний напрямок (контейнер 172.17.0.2 → example.com:80), друга — очікувана відповідь, і в ній видно NAT: сервер відповідає не контейнеру, а на **192.168.65.3** — адресу VM після MASQUERADE. Conntrack пам'ятає цю відповідність і переписує адреси у зворотних пакетах. `TIME_WAIT` — стан після закриття TCP-сесії, `[ASSURED]` — обмін пройшов в обидва боки, запис захищений від витіснення. Всього в таблиці було 1480 записів. Це той самий механізм, завдяки якому мій домашній роутер знає, кому з 192.168.8.0/24 віддати зворотний пакет.

Приватні хопи `10.12.18.1` і `10.1.1.3` у traceroute з ДЗ 4 сумісні зі схемою CGNAT, але самі по собі їм не доводять: для підтвердження треба порівняти WAN-адресу роутера з публічною `91.226.253.120`. Повний вивід: [docs/conntrack-output.txt](#).

7. Опціонально: dig AAAA + curl -6 проти A-запису

```
$ dig AAAA google.com +noall +answer
google.com. 257 IN AAAA 2a00:1450:4025:800::8a    (та ще 3 адреси ::64/::65/::66)

$ dig A google.com +noall +answer
google.com. 35 IN A 142.250.109.113              (та ще 5 адрес .100—.139)
```

Порівняння: A-записи — шість 32-бітних адрес; AAAA — чотири 128-бітні, всі в одній /64-мережі дата-центру Google (`::` у записі стискає нульові групи). Обидва запити обробив резолвер провайдера `10.255.255.1` за 7–10 мс — **сам запит AAAA їде по IPv4-транспорті**, IPv6-зв'язність для цього не потрібна. (Після перебудови мережі відповіді вказують на інший кластер Google — `4025:800` замість `4001:c0f`, адреси `.109.x` замість `.110.x`: DNS-балансування видало інший фронтенд, це нормально.)

А от скористатися AAAA-відповіддю не вийде: `curl -6 google.com` у цьому середовищі отримує IPv4-mapped `::ffff:142.250.109.100`, а пряме з'єднання з реальною адресою `2a00:1450:4001:c0f::64` падає з `No route to host`. Головний висновок: **AAAA-запити працюють, але транспортної IPv6-зв'язності немає**; DNS і маршрутизація — незалежні речі. Повні виводи: [docs/dig-aaaa-output.txt](#).